SuperAgents Video Demo Script

CloudSmith, Hobbes, and BigWeaver

Total Duration: 3 minutes
Format: Brief intro (30s) + Three independent 50-second segments

---

SCENARIO OPTIONS

Choose one scenario for your demo. All three follow the same script structure with context-specific details.

SCENARIO 1: Oktank Events - Conference Platform

- **Product:** Oktank Events - a trade conference platform serving 10,000 attendees across 50 conferences annually
- **Architecture:** Microservices with different teams maintaining components in different repos
- **New Feature:** Attendee networking capabilities - allowing attendees to connect with speakers and each other
- **Repos Affected:** web app, admin dashboard, user service, notification service, analytics, shared libraries
- **How the feature spans repos:**
  - Web app: Networking UI and connection interface
  - Admin dashboard: Moderation tools for connections
  - User service: Connection APIs and relationship management
  - Notification service: Connection request notifications
  - Analytics: Network activity tracking
  - Shared libraries: Connection data models
- **Performance Issue:** Session search latency (8s → 400ms after optimization)
- **Security Vulnerabilities:** IDOR in connections, command injection in message search, JWT vulnerabilities

SCENARIO 2: Oktank CRM - Enterprise CRM

- **Product:** Oktank CRM - an enterprise CRM managing 50,000 sales reps across 200 enterprise customers
- **Architecture:** Monorepo with microservices for different business domains (sales, marketing, support, analytics)
- **New Feature:** AI-powered lead scoring - automatically scoring and prioritizing leads for sales teams
- **Repos Affected:** web frontend, API gateway, lead service, email service, analytics engine, shared UI components
- **How the feature spans repos:**
  - Web frontend: Lead score display and filtering UI
  - API gateway: Score endpoints and rate limiting
  - Lead service: Scoring algorithm and lead ranking logic
  - Email service: Automated follow-up triggers based on scores

    - Analytics engine: Score performance tracking and model training
    - Shared UI components: Score visualization widgets
- **Performance Issue:** Lead dashboard loading time (12s → 600ms after caching optimization)
- **Security Vulnerabilities:** SQL injection in lead filters, broken access control on customer data, API rate limit bypass

SCENARIO 3: Oktank Docs - Desktop Productivity Suite

- **Product:** Oktank Docs - a cross-platform desktop productivity app with 100,000 daily active users
- **Architecture:** Electron-based desktop app with cloud sync services and plugin ecosystem
- **New Feature:** Real-time collaboration - allowing multiple users to edit documents simultaneously
- **Repos Affected:** desktop app (Electron), sync service, collaboration server, plugin SDK, web client, shared protocol library
- **How the feature spans repos:**
    - Desktop app: Live cursor rendering and presence UI
    - Sync service: Real-time change propagation
    - Collaboration server: WebSocket connections and conflict resolution
    - Plugin SDK: Collaboration APIs for third-party plugins
    - Web client: Browser-based collaboration interface
    - Shared protocol library: Operational transform definitions
- **Performance Issue:** Document sync delays (5s → 250ms after WebSocket optimization)
- **Security Vulnerabilities:** XSS in shared documents, insecure local storage of credentials, plugin sandbox escape

---

VIDEO 1: BigWeaver - Enterprise-Scale Development (50 seconds)

Script Template (adapt based on chosen scenario):

Voiceover: "Meet **[PRODUCT_NAME]**. The product team wants to add **[NEW_FEATURE]**. They create an Epic in Jira describing the requirements and assign it to BigWeaver."

Visual: [PRODUCT_NAME] logo and dashboard → Jira epic being assigned to BW

Voiceover: "BigWeaver reviews the request, asks clarifying questions, then pulls API specs from Confluence and analyzes the codebase."

Visual: Short Jira exchange between dev and BW → Confluence docs being accessed

Voiceover: "BW determines it needs to make changes across 6 repositories: **[REPO_LIST]**. It applies the latest coding standards, learning from previous implementations to maximize code quality and consistency."

Visual: GitHub organization view with animation connecting the 6 targeted repos

Voiceover: "BigWeaver shares its execution plan in Jira and starts work. The team gets real-time updates in Slack whenever they need them."

Visual: Execution plan posted to Jira → Concurrent tasks starting in Gaia interface → Slack progress update

Voiceover: "BigWeaver is asynchronously implementing changes, building test cases, and validating its own work across all repos simultaneously."

Visual: GitHub branches being created in parallel → Tasks checking off in Gaia → Test suites running

Voiceover: "The result? **[FEATURE_COMPONENTS]**. All happening in parallel across repos. Six GitHub pull requests, 47 files changed, all tests passing. BigWeaver updates Jira, posts to Slack, and notifies the team. Ready to deploy. This isn't code completion - this is enterprise-scale feature development."

Visual: BigWeaver updates Jira tickets → Slack notification → 6 GitHub PRs with all checks passing ✓

Scenario-Specific Scripts:

Oktank Events Version:

"Meet Oktank Events - a conference platform serving 10,000 attendees. The product team wants to add attendee networking capabilities. They create an Epic in Jira and assign it to BigWeaver. BW reviews the request, pulls API specs from Confluence, and determines the feature needs changes across 6 repositories: web app for the UI, admin dashboard for moderation, user service for connection APIs, notification service for alerts, analytics for tracking, and shared libraries for data models. It applies the latest coding standards, learning from previous implementations. BigWeaver shares its execution plan and starts work. The team gets real-time updates in Slack. BigWeaver is asynchronously implementing the networking feature across all repos, building test cases, and validating its work. Six GitHub pull requests, 47 files changed, all tests passing. Six repos, all coordinated. Ready to deploy. This isn't code completion - this is enterprise-scale feature development."

Oktank CRM Version:

"Meet Oktank CRM - an enterprise CRM managing 50,000 sales reps. The product team wants to add AI-powered lead scoring. They create an Epic in Jira and assign it to BigWeaver. BW reviews the request, pulls API specs from Confluence, and determines the feature needs changes across 6 repositories: web frontend for score display, API gateway for endpoints, lead service for the scoring algorithm, email service for automated follow-ups, analytics engine for model training, and shared UI components for score widgets. It applies the latest coding standards, learning from previous implementations. BigWeaver shares its execution plan and starts work. The team gets real-time updates in Slack. BigWeaver is asynchronously implementing the lead scoring feature across all repos, building test cases, and validating its work. Six GitHub pull requests, 52 files changed, all tests passing. Six repos, all coordinated. Ready to deploy. This isn't code completion - this is enterprise-scale feature development."

Oktank Docs Version:

"Meet Oktank Docs - a desktop productivity app with 100,000 daily active users. The product team wants to add real-time collaboration. They create an Epic in Jira and assign it to BigWeaver. BW reviews the request, pulls API specs from Confluence, and determines the feature needs changes across 6 repositories: desktop app for live cursors, sync service for change propagation, collaboration server for WebSocket connections, plugin SDK for collaboration APIs, web client for browser access, and shared protocol library for operational transforms. It applies the latest coding standards, learning from previous implementations. BigWeaver shares its execution plan and starts work. The team gets real-time updates in Slack. BigWeaver is asynchronously implementing the collaboration feature across all repos, building test cases, and

validating its work. Six GitHub pull requests, 43 files changed, all tests passing. Six repos, all coordinated. Ready to deploy. This isn't code completion - this is enterprise-scale feature development."

Visuals (adapt component names per scenario):

- **1:20-1:25:** BigWeaver reading from Jira epic + Confluence docs
- **1:25-1:45:** 6-panel split screen showing concurrent work across repos
- **1:45-1:55:** Progress visualization:
  - 6 repos updating simultaneously
  - File change counters incrementing
  - Test suites running in parallel
- **1:55-2:10:** Integration actions + GitHub PR reveal:
  - 6 GitHub PRs created with detailed descriptions
  - Jira tickets: "In Review" status
  - Slack: "#engineering - [Feature] PRs ready for review"
  - All checks passing ✓

Timing: 1:20 - 2:10

VIDEO 2: Hobbes - AI-Powered Penetration Testing (50 seconds)

Script Template:

Voiceover: "Same feature, security perspective. BigWeaver's PRs are deployed to staging, and Hobbes starts autonomous penetration testing. Traditional scanners would flag 52 issues - mostly noise. Hobbes? It's an AI agent that thinks like a security expert. It discovers the **[FEATURE]** endpoints, tests authentication flows, and actively exploits vulnerabilities. Finding one: **[VULN_1]**. Finding two: **[VULN_2]**. Finding three: **[VULN_3]**. Hobbes creates Jira security tickets for each finding, documents the exploits in Confluence, and alerts the team in Slack with severity levels. BigWeaver receives the security report and implements fixes across all 6 repos. New GitHub PRs created. Hobbes re-runs the test. All vulnerabilities patched. Updates Jira, posts to Slack: 'Security verified.' The loop closes. No security team needed."

Scenario-Specific Scripts:

Oktank Events Version:

"Same feature, security perspective. BigWeaver's networking PRs are deployed to staging, and Hobbes starts autonomous penetration testing. Traditional scanners would flag 52 issues - mostly noise. Hobbes? It's an AI agent that thinks like a security expert. It discovers the networking endpoints, tests authentication flows, and actively exploits vulnerabilities. Finding one: Insecure Direct Object Reference - accessing other users' connections by manipulating IDs. Finding two: Command Injection in the message search - executing arbitrary commands. Finding three: JWT vulnerabilities - forging authentication tokens. Hobbes creates Jira security tickets for each finding, documents the exploits in Confluence, and alerts the team in Slack with severity levels. BigWeaver receives the security report and implements fixes across all 6 repos. New GitHub PRs created. Hobbes re-runs the test. All vulnerabilities patched. Updates Jira, posts to Slack: 'Security verified.' The loop closes. No security team needed."

Oktank CRM Version:

"Same feature, security perspective. BigWeaver's lead scoring PRs are deployed to staging, and Hobbes starts autonomous penetration testing. Traditional scanners would flag 48 issues - mostly noise. Hobbes? It's an AI agent that thinks like a security expert. It discovers the lead scoring endpoints, tests authentication flows, and actively exploits vulnerabilities. Finding one: SQL Injection in lead filters - extracting customer data from other accounts. Finding two: Broken access control - sales reps accessing leads outside their territory. Finding three: API rate limit bypass - automated scraping of lead data. Hobbes creates Jira security tickets for each finding, documents the exploits in Confluence, and alerts the team in Slack with severity levels. BigWeaver receives the security report and implements fixes across all 6 repos. New GitHub PRs created. Hobbes re-runs the test. All vulnerabilities patched. Updates Jira, posts to Slack: 'Security verified.' The loop closes. No security team needed."

Oktank Docs Version:

"Same feature, security perspective. BigWeaver's collaboration PRs are deployed to staging, and Hobbes starts autonomous penetration testing. Traditional scanners would flag 55 issues - mostly noise. Hobbes? It's an AI agent that thinks like a security expert. It discovers the collaboration endpoints, tests authentication flows, and actively exploits vulnerabilities. Finding one: XSS in shared documents - injecting malicious scripts into collaborative workspaces. Finding two: Insecure local storage - extracting user credentials from Electron app storage. Finding three: Plugin sandbox escape - breaking out of the plugin environment to access system resources. Hobbes creates Jira security tickets for each finding, documents the exploits in Confluence, and alerts the team in Slack with severity levels. BigWeaver receives the security report and implements fixes across all 6 repos. New GitHub PRs created. Hobbes re-runs the test. All vulnerabilities patched. Updates Jira, posts to Slack: 'Security verified.' The loop closes. No security team needed."

Visuals:

- **2:10-2:20:** Hobbes dashboard showing autonomous testing in progress (PLANNING → TESTING → TESTED)
- **2:20-2:35:** Split screen comparison:
  - Traditional scanner: 52 findings (static analysis, false positives)
  - Hobbes: 3 critical findings (active exploitation, real vulnerabilities)
- **2:35-2:50:** Three vulnerability demonstrations with integrations:
  - Finding 1 → Jira ticket "SEC-101: Critical [vulnerability type]"
  - Finding 2 → Confluence doc with PoC
  - Finding 3 → Slack alert "#security-critical"
- **2:50-3:00:** Integration loop completion:
  - Hobbes → BigWeaver handoff (security report)
  - Fixes applied, new GitHub PRs created
  - Hobbes re-tests: All exploits blocked ✓
  - Jira: All tickets resolved
  - Slack: "✓ Security verification complete"

Timing: 2:10 - 3:00

VIDEO 3: CloudSmith - Infrastructure Intelligence (50 seconds)

Script Template:

Voiceover: "**[TRIGGER_EVENT]**. **[PRODUCT_NAME]**'s **[COMPONENT]** is slowing down - **[PROBLEM_DESCRIPTION]**. The on-call engineer gets paged, but CloudSmith is already working. It's analyzed CloudWatch metrics, **[DATA_SOURCE]** - everything. The diagnosis? **[ROOT_CAUSE]**. CloudSmith's recommendation: **[SOLUTION]**. Impact: **[IMPROVEMENT]**. CloudSmith creates a Jira ticket with the full analysis, posts the architecture diagram to Confluence, and sends a Slack alert to the team. The implementation plan is ready before the engineer finishes their coffee."

Scenario-Specific Scripts:

Oktank Events Version:

"Conference season hits. Oktank Events' API is slowing down - session searches taking 8 seconds, attendees frustrated. The on-call engineer gets paged, but CloudSmith is already working. It's analyzed CloudWatch metrics, RDS query patterns, API Gateway logs - everything. The diagnosis? Every search hits the database with full table scans. No caching, no indexes. CloudSmith's recommendation: add ElastiCache and optimize queries. Impact: 95% latency reduction, from 8 seconds to 400ms. That's 20x faster searches. CloudSmith creates a Jira ticket with the full analysis, posts the architecture diagram to Confluence, and sends a Slack alert to the team. The implementation plan is ready before the engineer finishes their coffee."

Oktank CRM Version:

"Quarter-end rush hits. Oktank CRM's lead dashboard is crawling - taking 12 seconds to load, sales reps complaining. The on-call engineer gets paged, but CloudSmith is already working. It's analyzed CloudWatch metrics, PostgreSQL slow query logs, Redis cache patterns - everything. The diagnosis? Complex lead scoring queries running on every page load. No result caching, inefficient joins. CloudSmith's recommendation: implement Redis caching layer and optimize database indexes. Impact: 95% latency reduction, from 12 seconds to 600ms. That's 20x faster dashboards. CloudSmith creates a Jira ticket with the full analysis, posts the architecture diagram to Confluence, and sends a Slack alert to the team. The implementation plan is ready before the engineer finishes their coffee."

Oktank Docs Version:

"Peak usage hours hit. Oktank Docs' sync service is lagging - document changes taking 5 seconds to propagate, users frustrated. The on-call engineer gets paged, but CloudSmith is already working. It's analyzed CloudWatch metrics, WebSocket connection logs, DynamoDB performance data - everything. The diagnosis? Polling-based sync with inefficient database queries. No real-time push, no connection pooling. CloudSmith's recommendation: migrate to WebSocket-based push notifications and optimize DynamoDB queries. Impact: 95% latency reduction, from 5 seconds to 250ms. That's 20x faster sync. CloudSmith creates a Jira ticket with the full analysis, posts the architecture diagram to Confluence, and sends a Slack alert to the team. The implementation plan is ready before the engineer finishes their coffee."

Visuals:

- **0:30-0:35:** Phone alert + CloudSmith dashboard analyzing
- **0:35-0:45:** Fast-paced analysis visualization:
  - CloudWatch metrics showing performance degradation
  - Database/service logs showing bottlenecks
  - Cost breakdown showing inefficiency
- **0:45-1:05:** Architecture comparison:
  - Current: Inefficient architecture (slow, expensive, unscalable)
  - Proposed: Optimized architecture (fast, cheap, scalable)
- **1:05-1:20:** Integration actions + Results:
  - Jira ticket created: "PERF-1234: Optimize [component] latency"
  - Confluence page updated with architecture diagrams
  - Slack notification: "#engineering - Performance issue identified"
  - Results: [BEFORE] → [AFTER], load reduction

Timing: 0:30 - 1:20

TECHNICAL NOTES FOR PRODUCTION

Demo Environment Setup (adapt per scenario):

Oktank Events:

- Pre-populate with realistic conference data (45 sessions, 60 speakers)
- Stage networking feature components
- Prepare performance metrics showing 8s → 400ms improvement

Oktank CRM:

- Pre-populate with realistic CRM data (1000 leads, 50 sales reps)
- Stage lead scoring and automation components
- Prepare performance metrics showing 12s → 600ms improvement

Oktank Docs:

- Pre-populate with realistic documents and collaboration sessions
- Stage real-time collaboration components
- Prepare performance metrics showing 5s → 250ms improvement

All Scenarios:

- Have CloudSmith analysis pre-run for smooth demo
- Stage BigWeaver PRs across 6 repos for quick reveal
- Prepare Hobbes findings in advance

Visual Assets Needed:

- Agent logos (CloudSmith, BigWeaver, Hobbes)
- Product screenshots (before/after feature)
- Architecture diagrams (6-repo structure)

- Dashboard mockups with real metrics
- Code editor screens with syntax highlighting
- GitHub PR interface (6 simultaneous PRs)
- Performance comparison charts
- **Integration UI mockups:**
  - Jira tickets being created/updated
  - Confluence pages with architecture diagrams
  - Slack notifications in #engineering and #security-critical channels
  - GitHub PR descriptions with agent-generated content

KEY MESSAGES TO EMPHASIZE

1. **CloudSmith:** Proactive infrastructure intelligence that identifies performance bottlenecks
2. **BigWeaver:** Autonomous code transformation across multiple repositories simultaneously
3. **Hobbes:** Security that thinks like an engineer, not a scanner
4. **Agent Integration:** The agents work together seamlessly across complex architectures
5. **Tool Integration:** Native integration with Jira, Confluence, GitHub, and Slack - agents work within your existing workflow
6. **Business Impact:** Real performance improvements and time reductions
7. **Developer Experience:** Focus on features, not infrastructure and security toil - agents handle the coordination

---

QUICK REFERENCE: Scenario Comparison

Element	Oktank Events	Oktank CRM	Oktank Docs
Industry	Events/Conferences	Sales/CRM	Productivity
Scale	10K attendees	50K sales reps	100K DAU
Feature	Networking	Lead Scoring	Collaboration
Perf Issue	8s → 400ms	12s → 600ms	5s → 250ms
Security 1	IDOR	SQL Injection	XSS
Security 2	Command Injection	Access Control	Credential Storage
Security 3	JWT Vuln	Rate Limit Bypass	Sandbox Escape